

Informática II

Introducción a microcontroladores (μC)

Gonzalo F. Perez Paina



Universidad Tecnológica Nacional
Facultad Regional Córdoba
UTN-FRC

– 2025 –

Programación en lenguaje C (PC / μ P)

- ▶ ¿Cuál es el primer programa realizado en “*Informática I*”?

Programación en lenguaje C (PC / μ P)

- ¿Cuál es el primer programa realizado en “*Informática I*”?

hola_mundo.c

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      printf("Hola mundo.\n");
6      return 0;
7  }
```

Programación en lenguaje C (PC / μ P)

- ¿Cuál es el primer programa realizado en “*Informática I*”?

hola_mundo.c

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      printf("Hola mundo.\n");
6      return 0;
7  }
```

¿Cómo será el programa “*Hola mundo*” para un Sistema Embebido (SE)?

Programación en lenguaje C (PC / μ P)

- ¿Cuál es el primer programa realizado en “*Informática I*”?

hola_mundo.c

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      printf("Hola mundo.\n");
6      return 0;
7  }
```

¿Cómo será el programa “*Hola mundo*” para un Sistema Embebido (SE)?

(<https://helloworldcollection.de/>)

Programación en lenguaje C (PC / μ P)

- ▶ ¿Qué otro programa simple recuerda?

Programación en lenguaje C (PC / μ P)

- ¿Qué otro programa simple recuerda?

suma1.c

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int entero1, entero2, suma;
6
7     printf("Ingrese el primer entero: ");
8     scanf("%d", &entero1);
9     printf("Ingrese el segundo entero: ");
10    scanf("%d", &entero2);
11
12    suma = entero1 + entero2;
13    printf("La suma es: %d\n", suma);
14    return 0;
15 }
```

Programación en lenguaje C (PC / μ P)

- ¿Qué otro programa simple recuerda?

suma1.c

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int entero1, entero2, suma;
6
7     printf("Ingrese el primer entero: ");
8     scanf("%d", &entero1);
9     printf("Ingrese el segundo entero: ");
10    scanf("%d", &entero2);
11
12    suma = entero1 + entero2;
13    printf("La suma es: %d\n", suma);
14    return 0;
15 }
```

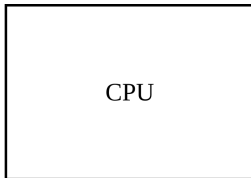
- ¿Cómo sería el programa “suma” en un SE?

¿Cómo funciona una PC (μ P) [repaso]

¿Qué función cumple una PC?

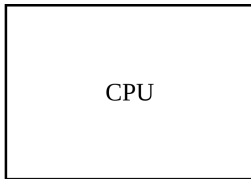
¿Cómo funciona una PC (μ P) [repaso]

Procesa datos



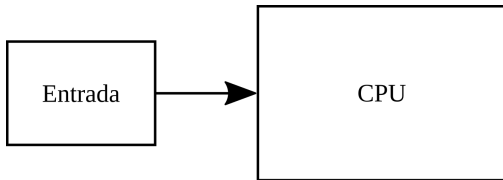
¿Cómo funciona una PC (μ P) [repaso]

¿De donde se obtienen los datos?



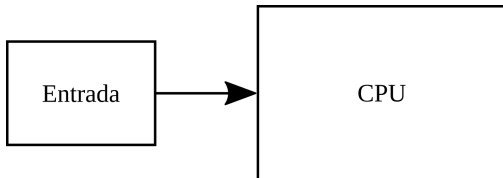
¿Cómo funciona una PC (μ P) [repaso]

¿De donde se obtienen los datos?



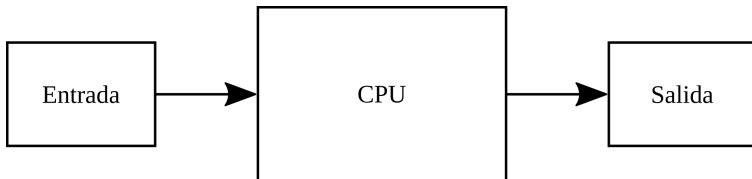
¿Cómo funciona una PC (μ P) [repaso]

¿Cómo pone a disposición el resultado?



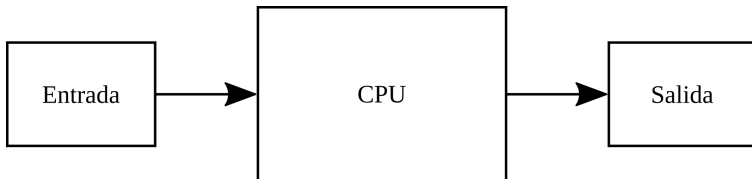
¿Cómo funciona una PC (μ P) [repaso]

¿Cómo pone a disposición el resultado?



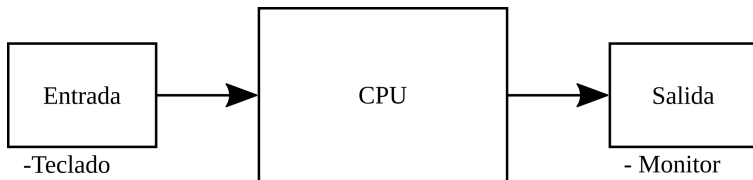
¿Cómo funciona una PC (μ P) [repaso]

¿Qué dispositivos de **entrada** y **salida** conoce?



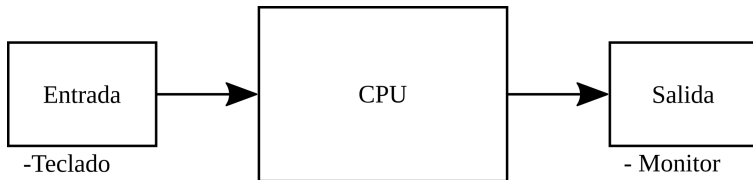
¿Cómo funciona una PC (μ P) [repaso]

¿Qué dispositivos de **entrada** y **salida** conoce?



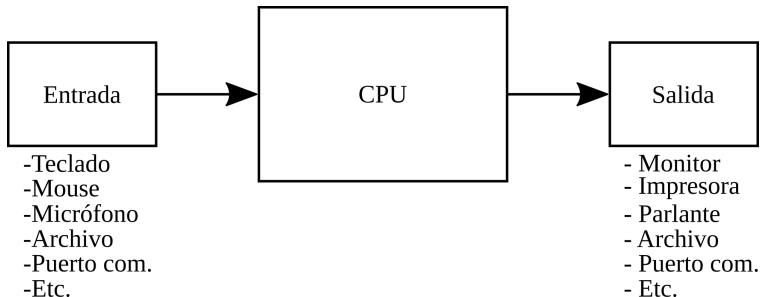
¿Cómo funciona una PC (μ P) [repaso]

Otros dispositivos de entrada/salida



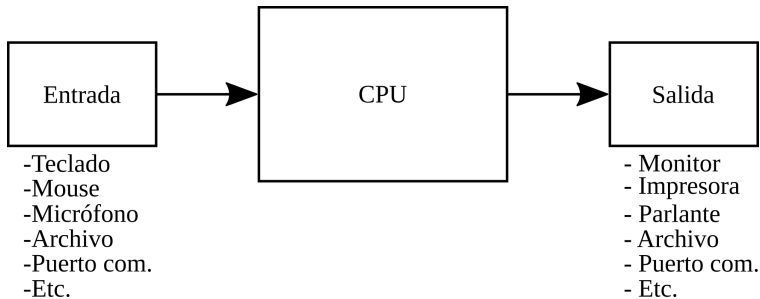
¿Cómo funciona una PC (μ P) [repaso]

Otros dispositivos de entrada/salida



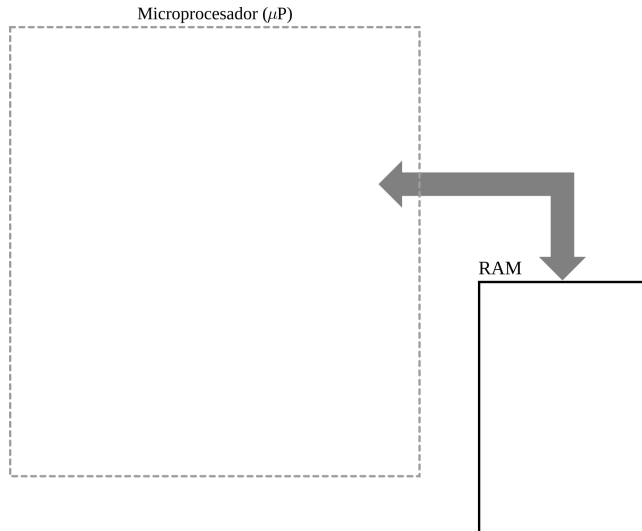
¿Cómo funciona una PC (μ P) [repaso]

Analice el ejemplo de suma dos números (programa en C)



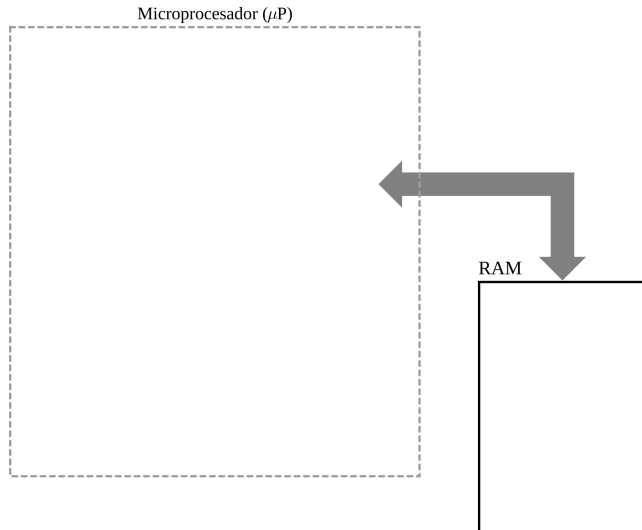
¿Cómo funciona una PC (μ P) [repaso]

¿Qué información almacena la memoria RAM?



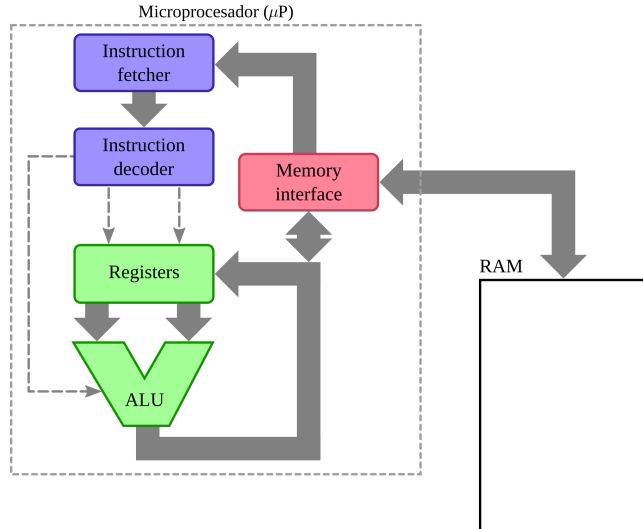
¿Cómo funciona una PC (μ P) [repaso]

¿Cuáles son los bloques principales de un μ P?



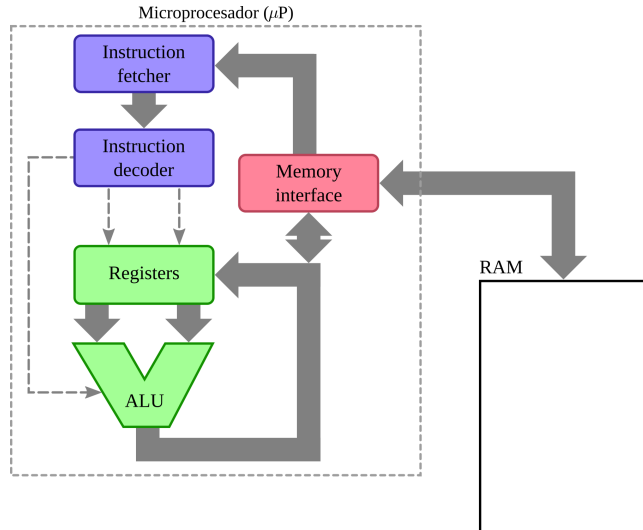
¿Cómo funciona una PC (μ P) [repaso]

¿Cuáles son los bloques principales de un μ P?



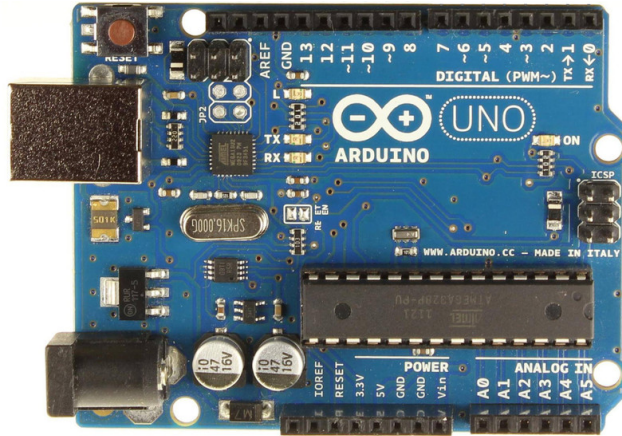
¿Cómo funciona una PC (μ P) [repaso]

¿Qué nombre recibe esta arquitectura?



Arduino UNO Rev3

Cuenta con un microcontrolador (μC) marca Microchip® (antiguamente Atmel®) modelo ATmega328.

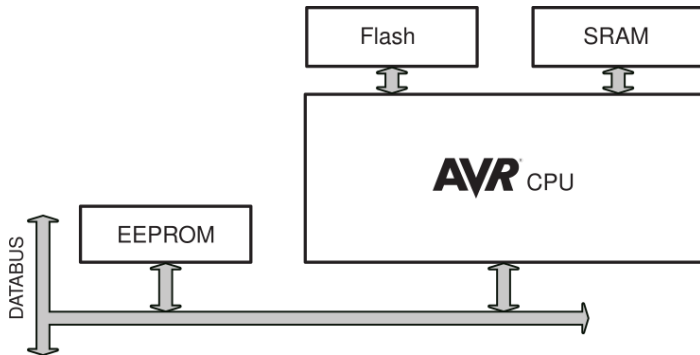


Arduino UNO Rev3 – Algunas características

Memoria flash	32 KB (0.5 KB bootloader)
SRAM	2 KB
EEPROM	1 KB
Arquitectura	Hardvard
Frecuencia de reloj	16 MHz

Arduino UNO Rev3 – Algunas características

Memoria flash	32 KB (0.5 KB bootloader)
SRAM	2 KB
EEPROM	1 KB
Arquitectura	Hardvard
Frecuencia de reloj	16 MHz



Arduino UNO Rev3 – Algunas características

Pines de E/S (I/O)	14 (6 PWM)
Pines de entrada analógica	6
Voltaje de operación	5V
Corriente por pin	40mA
Corriente por pin 3.3V	50mA

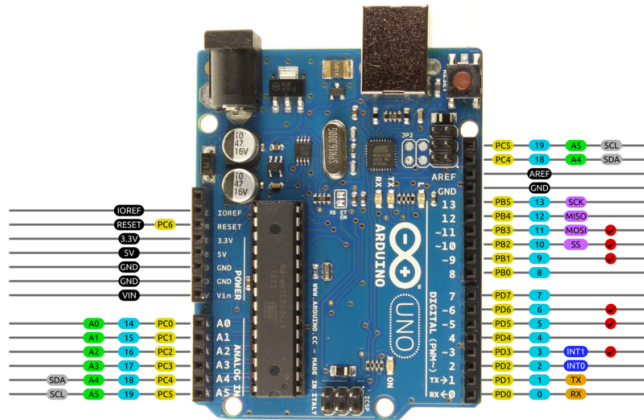
Arduino UNO Rev3 – Algunas características

Pines de E/S (I/O)	14 (6 PWM)
Pines de entrada analógica	6
Voltaje de operación	5V
Corriente por pin	40mA
Corriente por pin 3.3V	50mA

Algunos conceptos sobre sistemas embebidos:

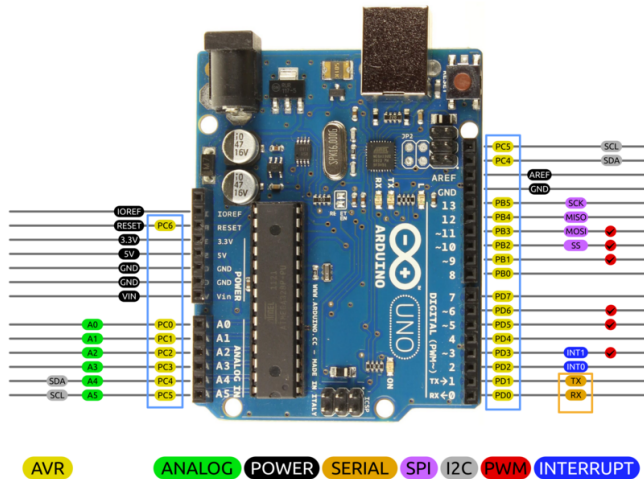
- ▶ Tipo de memoria: FLASH, SRAM, EEPROM (E²PROM)
- ▶ Arquitectura: Harvard vs. Von Neumann
- ▶ Entrada/salida de propósitos generales (GPIO)
- ▶ Modulación de ancho de pulso (PWM: Pulse Width Modulation)
- ▶ Conversor analógico a digital (ADC: Analog to Digital Converter)

Arduino UNO Rev3 – Periféricos (E/S digitales y UART)

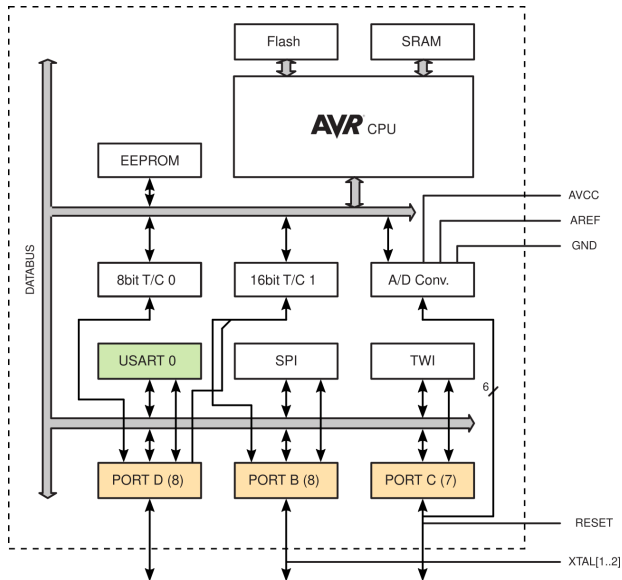


AVR DIGITAL ANALOG POWER SERIAL SPI I2C PWM INTERRUPT

Arduino UNO Rev3 – Periféricos (E/S digitales y UART)



El μ C ATmega328 – Diagrama en bloques



El μ C ATmega328 – Memorias

Tiene dos espacios de memorias diferentes (arquitectura Harvard):

1. una es la memoria de programa y
2. otra la memoria de datos.

El μ C ATmega328 – Memorias

Tiene dos espacios de memorias diferentes (arquitectura Harvard):

1. una es la memoria de programa y
2. otra la memoria de datos.

Memoria de programa (Flash)

- ▶ Almacena el programa, es de $16K \times 16\text{-bits}$ haciendo un total de 32KBytes ($32K \times 8\text{-bits}$).
- ▶ El contador de programa (PC: Program Counter) es de 14-bits lo que permite direccionar 16K ubicaciones de la memoria de programa.

El μ C ATmega328 – Memorias

Tiene dos espacios de memorias diferentes (arquitectura Harvard):

1. una es la memoria de programa y
2. otra la memoria de datos.

Memoria de programa (Flash)

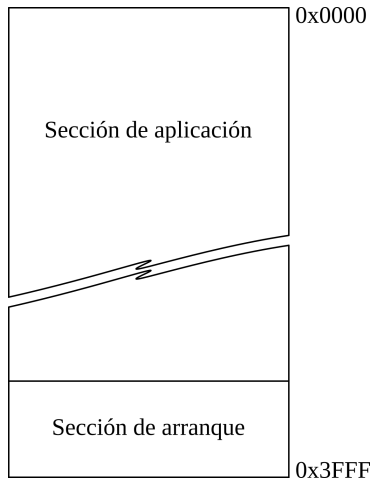
- ▶ Almacena el programa, es de $16K \times 16\text{-bits}$ haciendo un total de 32KBytes ($32K \times 8\text{-bits}$).
- ▶ El contador de programa (PC: Program Counter) es de 14-bits lo que permite direccionar 16K ubicaciones de la memoria de programa.

Memoria de programa (RAM)

La memoria RAM se organiza en diferentes secciones.

- ▶ Las primeras direcciones para acceder a los registros de propósito general y los de entrada-salida,
- ▶ las siguientes para almacenar las variables del programa.

El μ C ATmega328 – Memorias

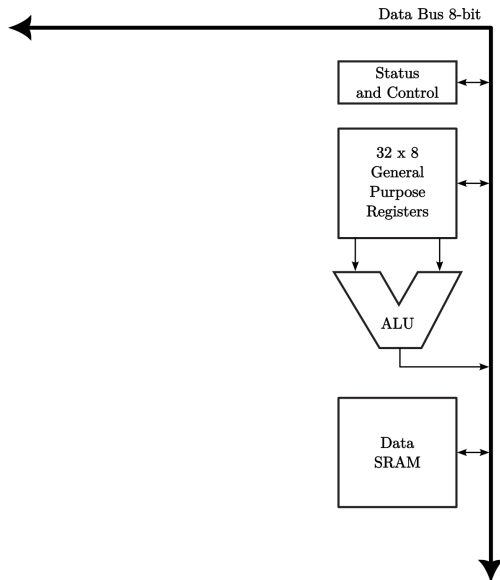


Mapa de mem. Flash

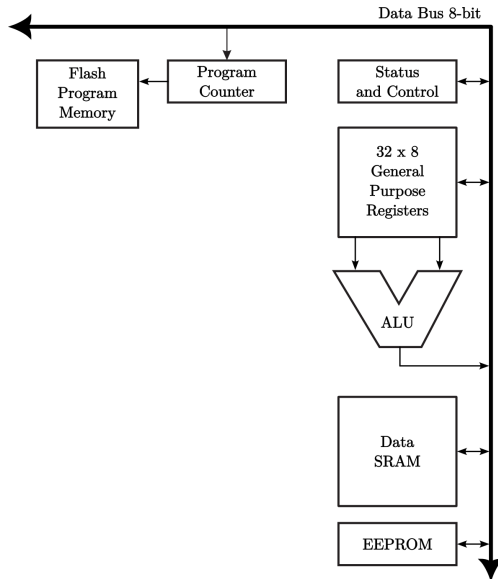
32 registros	0x0000 - 0x001F
64 registros de E/S	0x0020 - 0x005F
	0x0100
Memoria RAM gral (2048 x 8-bits)	0x08FF

Mapa de mem. RAM

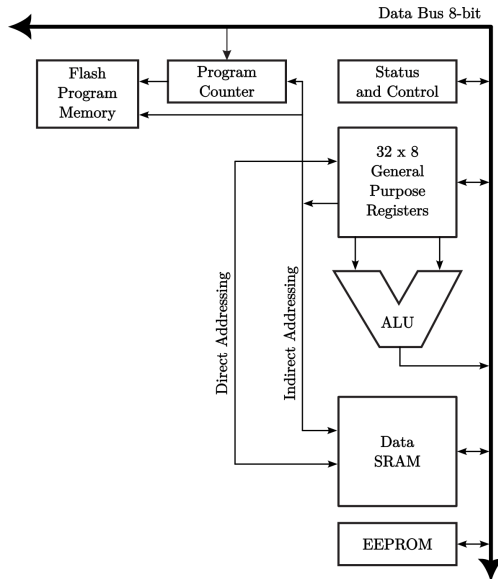
El μ C ATmega328 – Diagrama en bloques



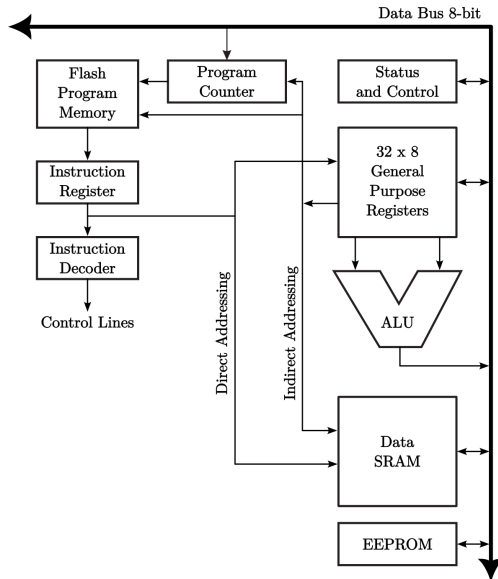
El μ C ATmega328 – Diagrama en bloques



El μ C ATmega328 – Diagrama en bloques

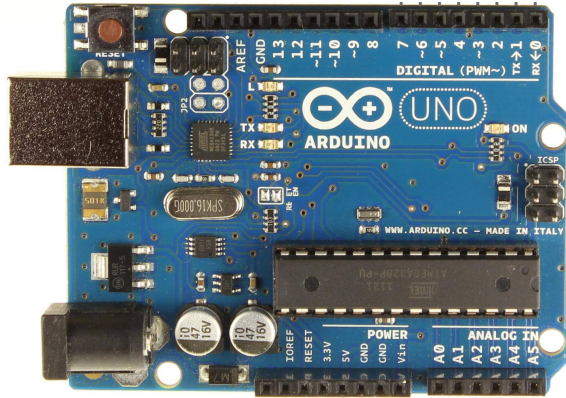


El μ C ATmega328 – Diagrama en bloques



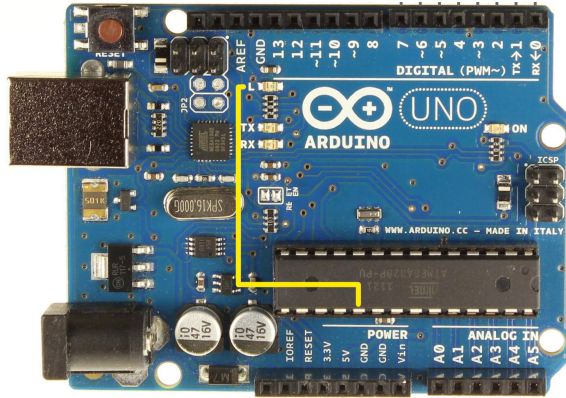
Programación en lenguaje C (μ C)

Programa “*Hola mundo*” en un SE $\rightarrow$ blinking LED.



Programación en lenguaje C (μ C)

Programa “*Hola mundo*” en un SE $\rightarrow$ blinking LED.



Pin 13 (Arduino) o PB5: conectado a LED (etiqueta L)

Programación en lenguaje C (μ C)

"Hola mundo" para el ATmega328 (blink.c)

```
1 #include <avr/io.h>
2
3
4
5 int main (void)
6 {
7     DDRB |= (1 << DDB5); /* Pin 5 de PORTB como salida */
8
9     while(1)
10    {
11        PORTB |= (1 << PORTB5); /* Pin 5 de PORTB en 1 */
12
13
14        PORTB &= ~(1 << PORTB5); /* Pin 5 de PORTB en 0 */
15
16    }
17    return 0;
18 }
```

Programación en lenguaje C (μ C)

"Hola mundo" para el ATmega328 (blink.c)

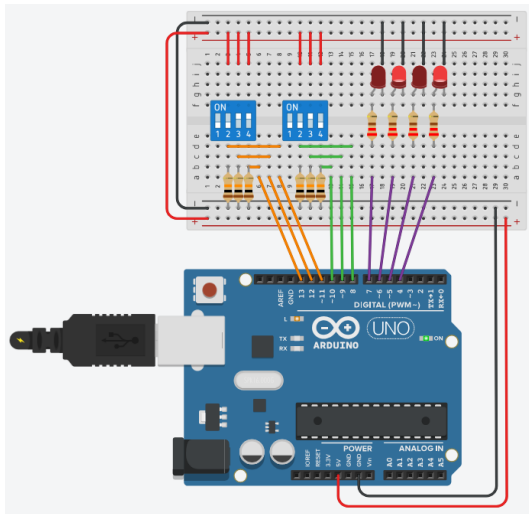
```
1 #include <avr/io.h>
2 #include <util/delay.h>
3 #define BLINK_DELAY_MS 1000
4
5 int main (void)
6 {
7     DDRB |= (1 << DDB5); /* Pin 5 de PORTB como salida */
8
9     while(1)
10    {
11        PORTB |= (1 << PORTB5); /* Pin 5 de PORTB en 1 */
12        _delay_ms(BLINK_DELAY_MS);
13
14        PORTB &= ~(1 << PORTB5); /* Pin 5 de PORTB en 0 */
15        _delay_ms(BLINK_DELAY_MS);
16    }
17    return 0;
18 }
```

Programación en lenguaje C (μ C)

¿Cómo implementar un programa “*suma*” (Arduino UNO)?

Programación en lenguaje C (μ C)

¿Cómo implementar un programa “*suma*” (Arduino UNO)?



Programación en lenguaje C (μ C) – Toolchain GCC para AVR

El Toolchain de GCC para arquitectura AVR está compuesto por:

`gcc`: el compilador de C y C++

`binutils`: colección de herramientas para la manipulación de archivos binarios

Programación en lenguaje C (μ C) – Toolchain GCC para AVR

El Toolchain de GCC para arquitectura AVR está compuesto por:

`gcc`: el compilador de C y C++

`binutils`: colección de herramientas para la manipulación de archivos binarios

`libc-avr`: subconjunto de la biblioteca estándar de C con funciones específicas para arquitectura AVR

`avrdude`: programa para escribir y leer la memoria Flash del μ C

Programación en lenguaje C (μ C) – Toolchain GCC para AVR

El Toolchain de GCC para arquitectura AVR está compuesto por:

`gcc`: el compilador de C y C++

`binutils`: colección de herramientas para la manipulación de archivos binarios

`libc-avr`: subconjunto de la biblioteca estándar de C con funciones específicas para arquitectura AVR

`avrdude`: programa para escribir y leer la memoria Flash del μ C

La instalación del Toolchain en Ubuntu se realiza instalando los siguientes paquetes: `gcc-avr`, `binutils-avr`, `avr-libc` y `avrdude`.

Programación en lenguaje C (μ C) – Toolchain GCC para AVR

El Toolchain de GCC para arquitectura AVR está compuesto por:

gcc: el compilador de C y C++

binutils: colección de herramientas para la manipulación de archivos binarios

libc-avr: subconjunto de la biblioteca estándar de C con funciones específicas para arquitectura AVR

avrdude: programa para escribir y leer la memoria Flash del μ C

La instalación del Toolchain en Ubuntu se realiza instalando los siguientes paquetes: **gcc-avr**, **binutils-avr**, **avr-libc** y **avrdude**.

- ▶ El compilador GCC para AVR (**avr-gcc**) es en realidad es un compilador cruzado (cross-compiler).
- ▶ Un compilador cruzado es aquel que genera código máquina para una arquitectura diferente a aquella en la cual se ejecuta el compilador.

Programación en lenguaje C (μ C) – Toolchain GCC para AVR

Construcción del programa y grabación en la memoria del μ C ATmega328 de la placa Arduino UNO:

Programación en lenguaje C (μ C) – Toolchain GCC para AVR

Construcción del programa y grabación en la memoria del μ C ATmega328 de la placa Arduino UNO:

1. Construcción del código fuente (.c) hasta la etapa de ensamblado, de la cual se obtiene un archivo objeto (.o):

```
avr-gcc -Os -DF_CPU=16000000UL -mmcu=atmega328p -c -o blink.o blink.c
```

Programación en lenguaje C (μ C) – Toolchain GCC para AVR

Construcción del programa y grabación en la memoria del μ C ATmega328 de la placa Arduino UNO:

1. Construcción del código fuente (.c) hasta la etapa de ensamblado, de la cual se obtiene un archivo objeto (.o):

```
avr-gcc -Os -DF_CPU=16000000UL -mmcu=atmega328p -c -o blink.o blink.c
```

2. Enlazado con bibliotecas, del cual se obtiene un archivo binario (.elf):

```
avr-gcc -mmcu=atmega328p blink.o -o blink.elf
```

Programación en lenguaje C (μ C) – Toolchain GCC para AVR

Construcción del programa y grabación en la memoria del μ C ATmega328 de la placa Arduino UNO:

1. Construcción del código fuente (.c) hasta la etapa de ensamblado, de la cual se obtiene un archivo objeto (.o):

```
avr-gcc -Os -DF_CPU=16000000UL -mmcu=atmega328p -c -o blink.o blink.c
```

2. Enlazado con bibliotecas, del cual se obtiene un archivo binario (.elf):

```
avr-gcc -mmcu=atmega328p blink.o -o blink.elf
```

3. Conversión del código binario (.elf) a formato objeto hexadecimal Intel (.hex)

```
avr-objcopy -O ihex -R .eeprom blink.elf blink.hex
```

Programación en lenguaje C (μ C) – Toolchain GCC para AVR

Construcción del programa y grabación en la memoria del μ C ATmega328 de la placa Arduino UNO:

1. Construcción del código fuente (.c) hasta la etapa de ensamblado, de la cual se obtiene un archivo objeto (.o):

```
avr-gcc -Os -DF_CPU=16000000UL -mmcu=atmega328p -c -o blink.o blink.c
```

2. Enlazado con bibliotecas, del cual se obtiene un archivo binario (.elf):

```
avr-gcc -mmcu=atmega328p blink.o -o blink.elf
```

3. Conversión del código binario (.elf) a formato objeto hexadecimal Intel (.hex)

```
avr-objcopy -O ihex -R .eeprom blink.elf blink.hex
```

4. Grabación en la memoria de programa del μ C

```
avrdude -F -V -c arduino -p ATMEGA328P -P /dev/ttyACM0 -b 115200  
-U flash:w:blink.hex
```

Programación en lenguaje C (μ C) – Makefile

```
1 PROG = blink
2 # Características del microcontrolador
3 MMCU = atmega328p
4 F_CPU = 16000000UL
5 # Toolchain
6 CC = avr-gcc
7 OBJCOPY = avr-objcopy
8 CFLAGS = -Wall -mmcu=$(MMCU)
9 # Targets
10 all: $(PROG).elf
11
12 $(PROG).o: $(PROG).c
13     $(CC) $(CFLAGS) -Os -DF_CPU=$(F_CPU) -c -o $(PROG).o $(PROG).c
14
15 $(PROG).elf: $(PROG).o
16     $(CC) $(CFLAGS) $(PROG).o -o $(PROG).elf
17
18 $(PROG).hex: $(PROG).elf
19     $(OBJCOPY) -O ihex -R .eeprom $(PROG).elf $(PROG).hex
20
21 .PHONY: clean
22 clean:
23     rm -f $(PROG).o $(PROG).elf $(PROG).hex
```